CNG 336 Introduction to Embedded Systems Development / EEE 347 Introduction to Microprocessors

LAB 01 [2.5%]

Atmega 128 Basic Assembly Language Programming

Student ID	Name	Surname
2584985	Zanyar	Erkozan

TASK 1

```
.include "m128def.inc"    // Include definitions for ATmega128 need this
.cseg                     // Code segment need this
.org 0x00                 // Program starts at address 0x00
LDI R16, 0x2A             // Load immediate 2A into R16
LDI R17, 0x5C             // Load immediate 5C into R17
ADD R16, R17              // Add R17 to R16 → result = 0x86
STS 0x0125, R16           // Store result the (0x86) into SRAM address 0x0125
LDS R18, 0x0125           // Load the value at 0x0125 back into R18
OUT SPH, R18              // Write the value (0x86) from R18 into SPH register
HERE: RJMP HERE           // Infinite loop (stop program execution)
```

Result:

Registers Before Execution:

Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x00
R21	0x00
R22	0x00
R23	0x00
R24	0x00
R25	0x00
R26	0x00
R27	0x00
R28	0x00
R29	0x00
R30	0x00
R31	0x00

Registers After Execution:

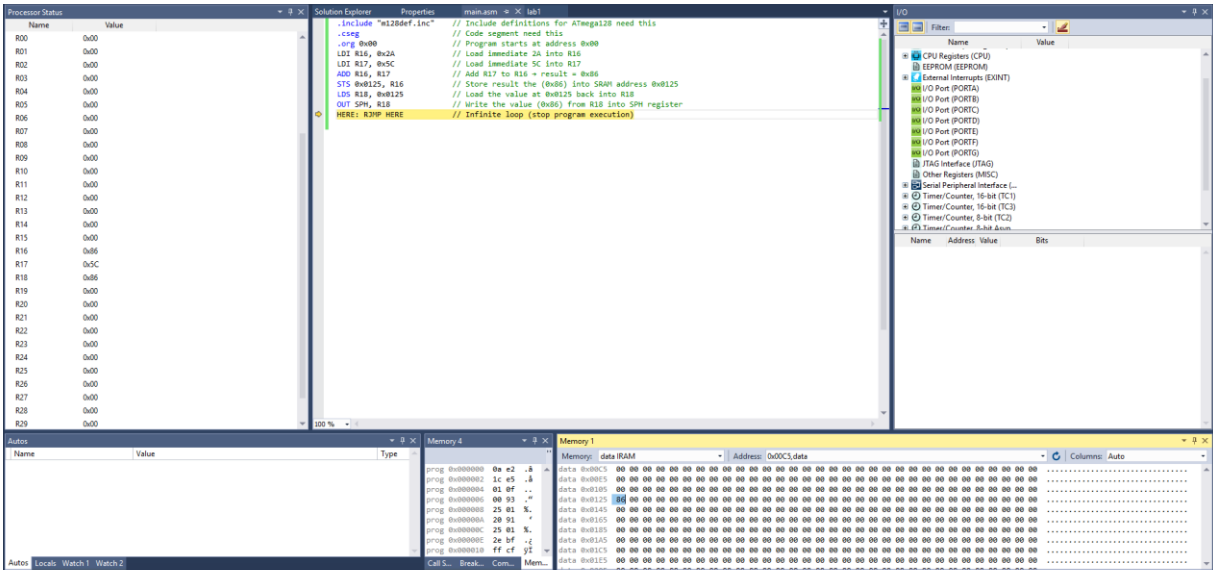
Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x86
R17	0x5C
R18	0x86
R19	0x00
R20	0x00
R21	0x00
R22	0x00
R23	0x00
R24	0x00
R25	0x00
R26	0x00
R27	0x00
R28	0x00
R29	0x00
R30	0x00
R31	0x00

Memory Before Execution: (Internal RAM)

```
Memory 1
Memory: data IRAM    Address: 0x00CE,data    Columns: Auto
data 0x00CE  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x00EB  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x0108  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x0125  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x0142  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x015F  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x017C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x0199  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x01B6  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
data 0x01D3  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .............................
```

Memory After Execution: (Internal RAM)

```
Memory: data IRAM    Address: 0x0077,data
data 0x0077  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x0094  00 06 00 00 00 00 00 20 00 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x00B1  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x00CE  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x00EB  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x0108  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x0125  86 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x0142  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x015F  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
data 0x017C  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```



Comment task 1:

General-purpose registers and I/O flags are at reset; SRAM location 0x0125 is not yet written; SPH retains its pre-execution value.

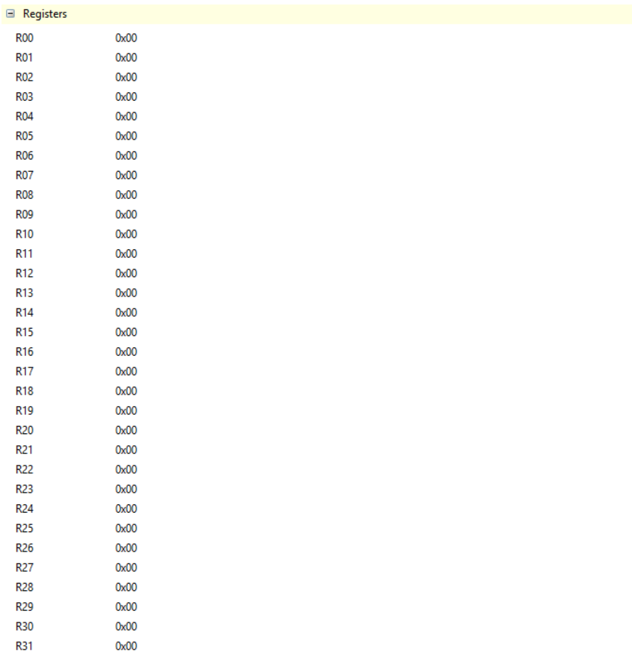
After computing and storing the sum, 0x86 value to SRAM 0x125, the same byte is loaded and written to the SPH I/O register; r16 and SPH both read 0x86, and SRAM[0x0125]=0x86, demonstrating correct addition, data transfer, and register update.

Task 2

```
.include "m128def.inc"

.cseg

.org 0x0000


eor  r1, r1         ; Clear r1 → used later to clear SREG

out  SREG, r1       ; Initialize SREG to 0

ldi  r16, 0x09      ; Load minuend (0x09)

ldi  r17, 0x05      ; Load subtrahend (0x05)

sub  r16, r17       ; Perform subtraction (0x09 - 0x05 = 0x04)

in   r20, SREG      ; Copy SREG content into r20

out  SREG, r1       ; Clear SREG after storing

nop                 ; Stop here for simulation
```

Result:

Registers Before Execution:



Registers	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x00
R21	0x00
R22	0x00
R23	0x00
R24	0x00
R25	0x00
R26	0x00
R27	0x00
R28	0x00
R29	0x00
R30	0x00
R31	0x00

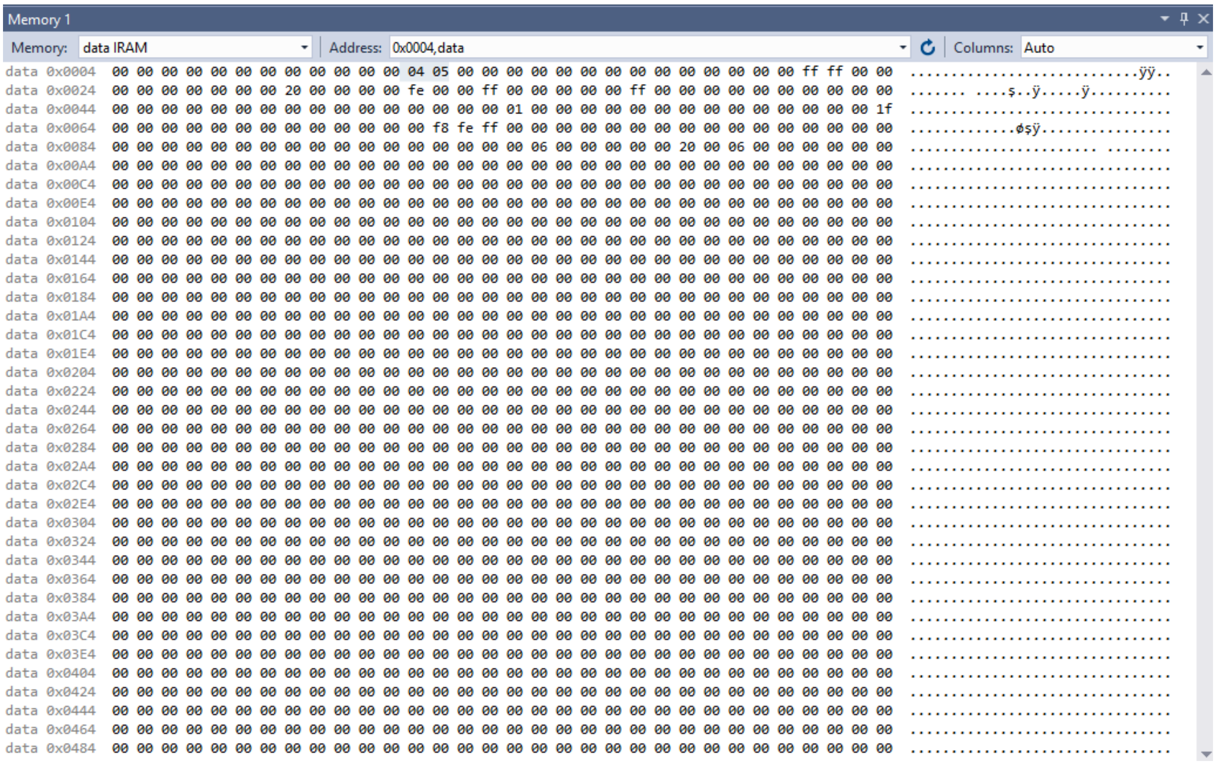
Registers After Execution:

```
⊟ Registers
R00    0x00
R01    0x00
R02    0x00
R03    0x00
R04    0x00
R05    0x00
R06    0x00
R07    0x00
R08    0x00
R09    0x00
R10    0x00
R11    0x00
R12    0x00
R13    0x00
R14    0x00
R15    0x00
R16    0x04
R17    0x05
R18    0x00
R19    0x00
R20    0x00
R21    0x00
R22    0x00
R23    0x00
R24    0x00
R25    0x00
R26    0x00
R27    0x00
R28    0x00
R29    0x00
R30    0x00
R31    0x00
```

Memory Before Execution: (Internal RAM)

```
Memory: data IRAM    Address: 0x0005,data    Columns: Auto
data 0x0005  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ff ff 00 00 00  ...........................ÿÿ...
data 0x0025  00 00 00 00 00 00 20 00 00 00 00 fe 00 00 ff 00 00 00 00 ff 00 00 00 00 00 00 00 00 00 00 00 00  ...... ....ş..ÿ.....ÿ...........
data 0x0045  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 1f 00  ................................
data 0x0065  00 00 00 00 00 00 00 00 00 00 00 00 f8 fe ff 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ............øşÿ.................
data 0x0085  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 06 00 00 00 00 00 20 00 06 00 00 00 00 00 00 00  ...................... .........
data 0x00A5  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................................
data 0x00C5  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................................
data 0x00E5  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................................
data 0x0105  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................................
data 0x0125  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................................
```

Memory After Execution: (Internal RAM):



Comment Task 2 :

After executing 0x09 – 0x05 = 0x04, all flags in SREG are 0. The Carry (C) bit is 0 because no borrow occurs (9 ≥ 5). The Half Carry (H) bit is 0 since no borrow happens between bit 3 and bit 4. The Zero (Z) bit is 0 because the result is not zero. The Negative (N) bit is 0 as the MSB of 0x04 is 0. The Overflow (V) bit is 0 since no signed overflow occurs. Hence, all C, H, Z, N, and V bits remain 0 before SREG is cleared.